The number of individual *dirty checks* is given by the following formula:

$$N = \sum_{k=1}^{n} p_k$$

- n - the number of managed entities
- p - the number of entity attributes.

Even if only one entity attribute changed, Hibernate would still have to go through all managed entities in the current Persistence Context. If the number of managed entities is fairly large, the default dirty checking mechanism might have a significant impact on CPU resources.

12.3.1.1 Controlling the Persistence Context size

Since the entity loading time snapshot is held separately, the Persistence Context requires twice as much memory to store a managed entity.

If the application developer does not need to update the selected entities, a read-only transaction will be much more suitable. From a Persistence Context perspective, a read-only transaction should use a read-only `Session`. By default, the `Session` loads entities in read-write mode, but this strategy can be customized either at the `Session` level or on a query basis:

```
//session-level configuration
Session session = entityManager.unwrap(Session.class);
session.setDefaultReadOnly(true);

//query-level configuration
List<Post> posts = entityManager.createQuery(
    "select p from Post p", Post.class)
.setHint(QueryHints.HINT_READONLY, true)
.getResultList();
```

When entities are loaded in read-only mode, there is no loading time snapshot being taken and the dirty checking mechanism is disabled for these entities.

This optimization addresses both memory and CPU resources. Since the persistent data snapshot is not stored anymore, the Persistence Context consumes half the memory required by a default read-write `Session`. Having fewer objects to manage, the Garbage Collector requires fewer CPU resources when it comes to reclaiming the memory of a closed Persistence Context. Flushing the Persistence Context is also faster and requires fewer CPU resources since the read-only entities are no longer dirty-checked.

When doing batch processing, it is very important to keep the Persistence Context size within bounds. One approach is to periodically *flush* and *clear* the Persistence Context. To avoid the issues associated with a long-running database transaction (e.g. locks being held for long periods of times, database memory consumption), the Java Persistence API allows a Persistence Context to span over multiple database transactions. This way, each batch job iteration clears the Persistence Context, commits the underlying transaction, and starts a new one for the next iteration.

```
EntityManager entityManager = null;
EntityTransaction transaction = null;

try {
    entityManager = entityManagerFactory().createEntityManager();

    transaction = entityManager.getTransaction();
    transaction.begin();

    for ( int i = 0; i < entityCount; ++i ) {
        if ( i > 0 && i % batchSize == 0 ) {
            entityManager.flush();
            entityManager.clear();

            transaction.commit();
            transaction.begin();
        }

        Post post = new Post( String.format( "Post %d", i + 1 ) );
        entityManager.persist( post );
    }

    transaction.commit();
} catch (RuntimeException e) {
    if ( transaction != null && transaction.isActive()) {
        transaction.rollback();
    }
    throw e;
} finally {
    if (entityManager != null) {
        entityManager.close();
    }
}
```

Another approach is to split the load into multiple smaller batch jobs and possibly process them concurrently. This way, long-running transactions are avoided and the Persistence Context has to manage only a limited number of entities.

The Persistence Context should be kept as small as possible. As a rule of thumb, only the entities that need to be modified should ever become managed. Read-only transactions should either use DTO projections or fetch entities in read-only mode.

12.3.2 Bytecode enhancement

Although Hibernate has supported bytecode enhancement for a long time, prior to Hibernate 5, the dirty checking mechanism was not taking advantage of this feature. Hibernate 5 has re-implemented the bytecode instrumentation mechanism, and now it is possible to avoid the reflection-based dirty checking mechanism. The bytecode enhancement can be done at compile-time, runtime or during deployment. The compile-time alternative is preferred for the following reasons:

- The enhanced classes can be covered by unit tests.
- The Java EE application server or the stand-alone container (e.g. Spring) can bootstrap faster because there's no need to instrument classes at runtime or deploy-time.
- Class loading issues are avoided since the application server does not have to take care of two versions of the same class (the original and the enhanced one).

The Hibernate tooling project comes with bytecode enhancement plugins for both Maven and Gradle. For Maven, the following plugin must be configured in the `pom.xml` file:

```
<plugin>
    <groupId>org.hibernate.orm.tooling</groupId>
    <artifactId>hibernate-enhance-maven-plugin</artifactId>
    <version>${hibernate.version}</version>
    <executions>
        <execution>
            <configuration>
                <enableDirtyTracking>true</enableDirtyTracking>
            </configuration>
            <goals>
                <goal>enhance</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

The bytecode enhancement plugin supports three instrumentation options which must be explicitly enabled during configuration:

- lazy initialization (allows entity attributes to be fetched lazily)
- dirty tracking (the entity tracks its own attribute changes)
- association management (allows automatic sides synchronization for bidirectional associations).

After the Java classes are compiled, the plugin goes through all entity classes and modifies their bytecode according to the instrumentation options chosen during configuration.

When enabling the *dirty tracking* option, Hibernate tracks attribute changes through the `$$_hibernate_tracker` attribute. Every setter method also calls the `$$_hibernate_trackChange` method to register the change.

```
@Transient
private transient DirtyTracker $$_hibernate_tracker;

public void $$_hibernate_trackChange(String paramString) {
    if (this.$$_hibernate_tracker == null) {
        this.$$_hibernate_tracker = new SimpleFieldTracker();
    }
    this.$$_hibernate_tracker.add(paramString);
}
```

Considering the following original Java entity class setter method:

```
public void setTitle(String title) {
    this.title = title;
}
```

Hibernate transforms it to the following bytecode representation:

```
public void setTitle(String title) {
    if(!EqualsHelper.areEqual(this.title, title)) {
        this.$$_hibernate_trackChange("title");
    }
    this.title = title;
}
```

When the application developer calls the `setTitle` method with an argument that differs from the currently stored `title`, the change is going to be recorded in the `$$_hibernate_tracker` class attribute.

During flushing, Hibernate inspects the `$$_hibernate_hasDirtyAttributes` method to validate if an entity was modified. The `$$_hibernate_getDirtyAttributes` method returns the names of all changed attributes.

```
public boolean $$_hibernate_hasDirtyAttributes() {
    return $$_hibernate_tracker != null && !$$_hibernate_tracker.isEmpty();
}

public String[] $$_hibernate_getDirtyAttributes() {
    if($$_hibernate_tracker == null) {
        $$_hibernate_tracker = new SimpleFieldTracker();
    }
    return $$_hibernate_tracker.get();
}
```

To validate the bytecode enhancement performance gain, the following test measures the dirty tracking time for 10, 20, 50, and 100 `Post` entity hierarchies (each `Post` is associated with one `PostDetails`, two `PostComment` and two `Tag` entities). Each iteration modifies six attributes: the `Post` title, the `PostDetails` creation date and owner, the `PostComment` review and the `Tag` name.

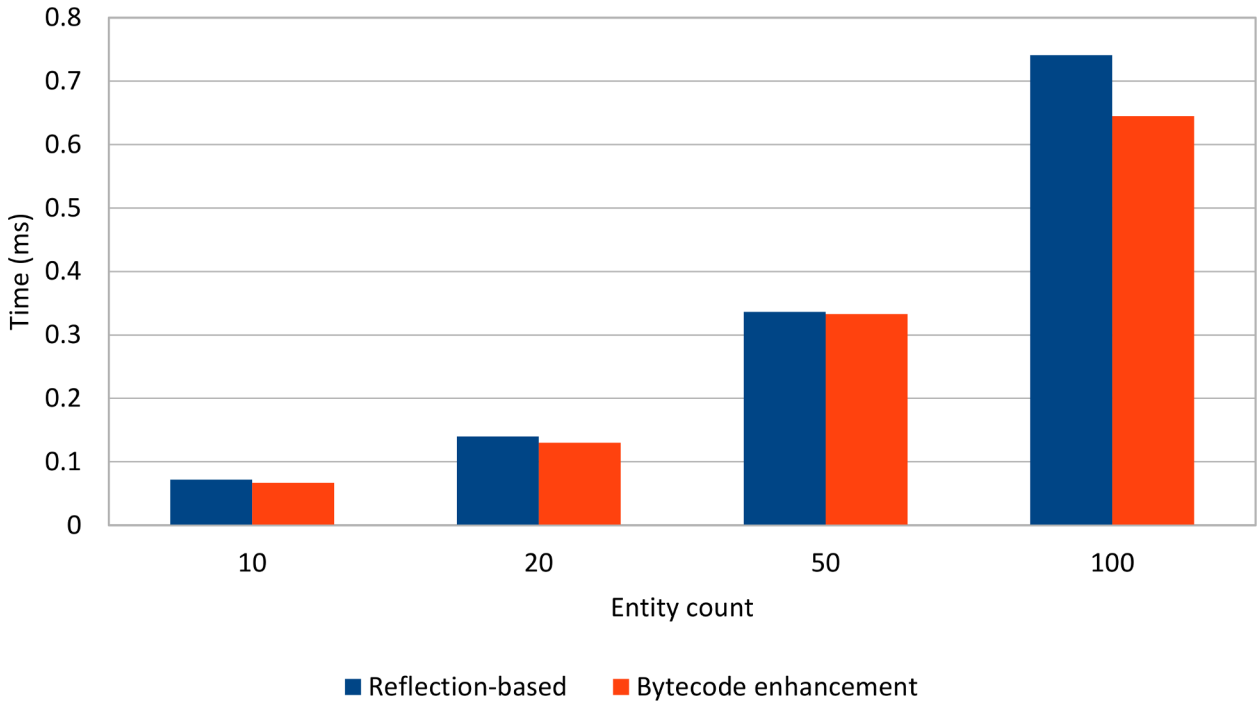

Figure 12.2: Bytecode enhancement performance gain

Both dirty checking mechanisms are very fast, and, compared to how much it takes to run a database query, the in-memory attribute modification tracking is insignificant. Up to 50 `Post` entities, the reflection-based and the bytecode enhancement dirty checking mechanisms perform comparably.